5. Keylime Registrar 컴포넌트 분석

5.1 Registrar의 역할과 책임

Keylime Registrar는 TPM 공개키 관리 및 에이전트 등록을 담당하는 핵심 컴포넌트입니다. Registrar는 다음과 같은 주요 역할을 수행합니다:

5.1.1 TPM 공개키 관리

- **EK(Endorsement Key) 저장 및 관리**: TPM 제조사가 생성한 고유 키 보관
- **AIK(Attestation Identity Key) 등록**: 증명용 신원 키 관리
- **IAK(Initial Attestation Key) 지원**: DevID 기반 등록 시 초기 증명 키 관리
- **IDevID(Initial Device Identity) 관리**: 초기 디바이스 신원 키 관리

5.1.2 에이전트 등록 프로세스 관리

- **에이전트 신원 검증**: TPM 키와 인증서 기반 신원 확인
- **MakeCredential/ActivateCredential 프로토콜**: TPM 2.0 표준 자격 증명 프로세스 구현
- **등록 상태 추적**: 에이전트의 등록 및 활성화 상태 관리

5.2 RegistrarData TypedDict 구조 분석

`keylime/registrar_client.py` 에서 정의된 `RegistrarData` 구조는 에이전트 정보를 표현하는 핵심 데이터 타입입니다:

```
class RegistrarData(TypedDict):

ip: Optional[str] # 에이전트 IP 주소
```

```
port: Optional[str] # 에이전트 포트 번호

regcount: int # 등록 횟수 카운터

mtls_cert: Optional[str] # 상호 TLS 인증서

aik_tpm: str # AIK TPM 공개키 (Base64 인코딩)

ek_tpm: str # EK TPM 공개키 (Base64 인코딩)

ekcert: Optional[str] # EK 인증서 (선택적)

provider_keys: NotRequired[Dict[str, str]] # 클라우드 제공자 키 (선택적)
```

5.2.1 필수 필드 검증

`MANDATORY_FIELDS` 상수로 정의된 필수 필드들:

```
MANDATORY_FIELDS = ["aik_tpm", "regcount", "ek_tpm", "ip", "port"]
```

5.3 getData() 함수 상세 분석

`keylime/registrar_client.py` 의 `getData()` 함수는 Tenant가 Registrar로부터 에이전트 정보를 조회하는 핵심 함수입니다:

5.3.1 함수 시그니처

```
def  getData(

registrar_ip: str,

registrar_port: str,

agent_id: str,
```

```
tls_context: Optional[ssl.SSLContext]

) -> Optional[RegistrarData]:
```

5.3.2 보안 검증 메커니즘

```
# AIK 키 정보 요청 시 반드시 서버 인증 TLS 필요

if not tls_context:

raise Exception("It is unsafe to use this interface to query AIKs without server
```

5.3.3 HTTP 응답 처리 로직

1. **404 응답**: 에이전트가 등록되지 않음
2. **200 응답**: 정상적인 데이터 반환
3. **기타 응답**: 예상치 못한 오류 상황

5.3.4 데이터 검증 및 변환

```
# 필수 필드 존재 여부 확인

if not check_mandatory_fields(response_body["results"]):

return None


# RegistrarData 구조로 변환

res: RegistrarData = {

"aik_tpm": r["aik_tpm"],
```

```
"regcount": r["regcount"],

"ek_tpm": r["ek_tpm"],

"ip": r["ip"],

"port": r["port"],

"mtls_cert": r.get("mtls_cert"),

"ekcert": r.get("ekcert"),

}
```

5.4 Agent 등록 프로세스에서의 EK 및 AK 검증

5.4.1 등록 프로세스 개요

Agent 등록은 `keylime/web/registrar/agents_controller.py` 의 `AgentsController` 에서 처리됩니다:

1. **POST /v2/agents/:agent_id** - 초기 등록 요청
2. **POST /v2/agents/:agent_id/activate** - 활성화 요청

5.4.2 초기 등록 단계 (create 메서드)

```
def  create(self, agent_id, **params):

agent = RegistrarAgent.get(agent_id) or RegistrarAgent.empty()

agent.update({"agent_id": agent_id, **params})

challenge = agent.produce_ak_challenge()

if not challenge or not agent.changes_valid:

self.respond(400, "Could not register agent with invalid data")
```

```
return
agent.commit_changes()
self.respond(200, "Success", {"blob": challenge})
```

5.4.3 EK 검증 프로세스

`keylime/models/registrar/registrar_agent.py` 의 `_check_ek()` 메서드에서 수행:

```
def _check_ek(self):
# EK TPM 공개키와 EK 인증서의 공개키 일치 여부 확인
self._check_key_against_cert("ek_tpm", "ekcert")
```

5.4.4 AK 챌린지 생성

`produce_ak_challenge()` 메서드에서 MakeCredential 프로토콜 실행:

```
def produce_ak_challenge(self):
if not self.ek_tpm or not self.aik_tpm:
return None
try:
result = Tpm.encrypt_aik_with_ek(self.agent_id, self.ek_tpm, self.aik_tpm)
if not result:
return None
except ValueError:
return None
```

```
(challenge, key) = result
self.change("key", key)
return challenge.decode("utf-8")
```

5.4.5 AK 응답 검증

`verify_ak_response()` 메서드에서 ActivateCredential 결과 검증:

```
def verify_ak_response(self, response):
expected_response = crypto.do_hmac(self.key.encode(), self.agent_id)
result = hmac.compare_digest(response, expected_response)
self.change("active", result)
return result
```

5.5 MakeCredential과 ActivateCredential 동작 원리

5.5.1 MakeCredential 프로세스

`keylime/tpm/tpm_util.py` 의 `makecredential()` 함수에서 구현:

```
def makecredential(ek_tpm: bytes, challenge: bytes, aik_name: bytes) -> bytes:
"""TPM_MakeCredential 구현
Parameters:
- ek_tpm: 마샬링된 TPMT_PUBKEY
- challenge: 랜덤 '패스워드'
- aik_name: 객체(AIK)의 이름
```

```
"""
public_key, hash_alg = tpm2_objects.pubkey_parms_from_tpm2b_public(ek_tpm)
hashfunc = tpm2_objects.HASH_FUNCS.get(hash_alg)
if not hashfunc:
raise ValueError(f"지원되지 않는 해시 알고리즘: {hash_alg:#x}")
```

5.5.2 RSA 기반 암호화

```
if  isinstance(public_key, RSAPublicKey):
random = os.urandom(hashfunc.digest_size)
secret = public_key.encrypt(
random,
padding.OAEP(
mgf=padding.MGF1(algorithm=hashfunc),
algorithm=hashfunc,
label=label_to_bytes("IDENTITY")
),
)
```

5.5.3 ECC 기반 암호화

```
elif  isinstance(public_key, EllipticCurvePublicKey):
```

```
random, secret = crypt_secret_encrypt_ecc(public_key, hashfunc)
```

5.5.4 자격 증명 생성

```
credentialblob = secret_to_credential(challenge, aik_name, random, ek_tpm, hashf

# tpm2-tools 특화 결과 형식 생성
hdr = struct.pack(">II", 0xBADCC0DE, 1)
tail = struct.pack(f">H{len(secret)}s", len(secret), secret)

return hdr + credentialblob + tail
```

5.5.5 ActivateCredential 검증 과정

1. **TPM에서 자격 증명 활성화**: Agent가 TPM2_ActivateCredential 명령 실행
2. **HMAC 계산**: 복호화된 챌린지로 HMAC 생성
3. **응답 검증**: Registrar가 예상 HMAC과 비교

```
# keylime/tpm/tpm_main.py의 encrypt_aik_with_ek 함수
@staticmethod
def  encrypt_aik_with_ek(uuid: str, ek_tpm: bytes, aik_tpm: bytes) -> Optional[T
aik_name = tpm2_objects.get_tpm2b_public_name(aik_tpm)
# 32바이트 랜덤 챌린지 생성
challenge_str = tpm_util.random_password(32)
```

```
challenge = challenge_str.encode()

logger.info("UUID %s에 대해 EK로 AIK 암호화 중", uuid)

# AES 키로 사용할 Base64 인코딩된 챌린지

key = base64.b64encode(challenge).decode("utf-8")

# MakeCredential 실행

credentialblob = tmp_util.makecredential(ek_tpm, challenge, bytes.fromhex(aik_na

keyblob = base64.b64encode(credentialblob)

return (keyblob, key)
```

5.6 보안 고려사항

5.6.1 TLS 필수 요구사항

- AIK 정보 조회 시 반드시 서버 인증 TLS 사용
- 상호 TLS(mTLS) 인증서 지원

5.6.2 타이밍 공격 방지

```
# HMAC 비교 시 타이밍 공격 방지

result = hmac.compare_digest(response, expected_response)
```

5.6.3 인증서 검증

- EK 인증서와 TPM 공개키 일치성 검증
- IAK/IDevID 인증서 신뢰 체인 검증

- ASN.1 DER 인코딩 규칙 준수 검사

이 분석을 통해 Keylime Registrar 컴포넌트가 TPM 기반 하드웨어 신뢰를 구축하는 핵심 역할을 수행하며, MakeCredential/ActivateCredential 프로토콜을 통해 안전한 에이전트 등록을 보장함을 확인할 수 있습니다.